

Jekyll Website Implementation Guide

Building Your Project Website with Jekyll Template

Overview

Learn how to create a professional website for your Big Data project using Jekyll and GitHub Pages

Getting Started with Jekyll Template

What You'll Learn

- **Template Setup:** Clone and configure the Jekyll template
- **Repository Management:** Setup GitHub repository and collaborators
- **Local Development:** Optional local development environment
- **GitHub Pages:** Deploy your website automatically

Prerequisites

- GitHub account

Step 1: Setting Up the Project Template

Using the Template Repository

1. **Navigate to:** `https://github.com/danielefadda/master-projects-template-2025`
2. **Click:** "Use this template" button (top right)
3. **Select:** "Create a new repository"
4. **Name format:** `g0-2025-website` (replace `g0` with your group number)
5. **Visibility:** Leave as Public
6. **Create:** Click "Create repository"

Understanding the Project Structure

Repository Structure

```
project-template/docs/  
├── _config.yml          # Basic site settings  
├── _data/  
│   └── settings.yml    # Visual config and content  
├── _pages/  
├── assets/  
├── _includes/  
└── _layouts/           # HTML layouts
```

Step 2: Collaborator Management

Adding Team Members

1. **Repository Settings:** Go to your repository settings
2. **Manage Access:** Click "Manage access"
3. **Invite Collaborators:** Add team members by GitHub username or email
4. **Required Teachers:** Always add: `danielefadda` and `Elecapp`

Collaboration Best Practices

- **Clear Communication:** Use GitHub issues for task tracking
- **Branch Protection:** Consider protecting main branch
- **Code Reviews:** Review each other's changes before merging



Step 3: Local Development Setup (Optional)

Ruby and Jekyll Installation

1. **Install Ruby:** Follow [official Jekyll guide](#)
2. **Clone Repository:** `git clone [your-repo-url]`
3. **Install Dependencies:** `bundle install`
4. **Launch Server:** `bundle exec jekyll serve`
5. **Access Site:** `http://127.0.0.1:4000/[repo-name]`

Benefits of Local Development

- **Faster Testing:** Immediate preview of changes
- **Offline Work:** No internet required for development
- **Advanced Editing:** Use your favorite IDE



Step 4: Editing Files - Three Methods

Method 1: Direct GitHub Editing

- **Simple Changes:** Edit files directly in browser
- **Quick Updates:** Perfect for content changes
- **No Setup:** No local installation required

Method 2: GitHub Codespaces

- **Advanced Editor:** Full IDE in browser
- **Syntax Highlighting:** Better code editing experience
- **Extensions:** Install Prettier for code formatting

Method 3: Local Development

- **Full Control:** Use any editor (VS Code, Sublime, etc.)
- **Git Integration:** Advanced version control
- **Offline Work:** No internet dependency

Step 5: Configuring _config.yml

```
# Repository settings
baseurl: "/g0-2025-website"  # Your repo name
url: "https://username.github.io"  # Your GitHub Pages URL

# Site information
title: "Your Project Title"
subtitle: "Project Subtitle"
description: "Brief project description"

# Repository links (footer)
github_repo:
  - name: "Group 0 - Website repository"
    url: "https://github.com/your-username/website-repo"
```



Step 6: GitHub Pages Deployment

Publishing Your Website

1. **Repository Settings:** Go to Settings → Pages
2. **Source Branch:** Select `main` branch
3. **Root Directory:** Choose `docs` folder
4. **Automatic Deployment:** Site updates on every push

Your Website URL

- **Format:** `https://username.github.io/repository-name/`
- **Example:** `https://johndoe.github.io/g0-2025-website/`

Step 6: GitHub Pages Deployment Tips

Deployment Tips

- **Build Time:** Allow 5-10 minutes for initial deployment
- **Updates:** Changes appear within minutes after push
- **Troubleshooting:** Check Actions tab for build errors

Step 7: Creating and Managing Pages

Page Structure

```
docs/_pages/  
├── index.md          # Homepage  
├── project.md        # Project description  
├── team.md           # Team members  
├── deliverables.md   # Results and downloads  
└── about.md          # Custom page (optional)
```

Step 7: Creating and Managing Pages

Creating a New Page

1. **Create File:** Add `.md` file in `_pages/` directory
2. **Front Matter:** Add YAML header with metadata
3. **Content:** Write in Markdown format
4. **URL:** File name becomes page URL

Step 8: Front Matter Configuration

Basic Front Matter

```
---  
layout: default  
title: "Page Title"  
subtitle: "Page Subtitle"  
header_type: hero  
header_img: assets/images/header.png  
vega: true  # Enable charts  
---
```

Step 8: Front Matter Configuration

Header Types

- **hero**: Full-width header with background image
- **base**: Standard header
- **post**: Blog-style header
- **image**: Image-focused header
- **splash**: Dramatic full-screen header

Step 9: Content Creation with Markdown

Markdown Basics

Heading 1

Heading 2

****Bold text****

Italic text

- Bullet list
- Another item

[Link text](https://example.com)

Enhanced Styling

This is a highlighted paragraph.

{: .lead}

This text appears in green.

{: .green}

Step 10: Adding Images

Image Management

- **Location:** Store images in `assets/images/`
- **Markdown Syntax:** `![Alt text](assets/images/image.jpg)`
- **HTML Tag:** ``

Image Galleries

```
<!-- Save images in assets/images/gallery_name/ -->
{% include_cached snippets/masonry.html internal="gallery_name" %}
```

Gallery Configuration

```
# In _config.yml
defaults:
  - scope:
      path: "assets/images/gallery_name"
    values:
      image_col: gallery_name
```

Step 11: Embedding Videos

YouTube Integration

```
<!-- Video shortcode -->
{% include video id="VIDEO_ID" provider="youtube" %}
```

Video Providers

- **YouTube:** Most common choice
- **Vimeo:** Professional videos
- **Dailymotion:** Alternative platform

Step 12: Adding Icons and Visual Elements

Font Awesome Icons

```
<i class="fa-solid fa-pen-nib"></i> Text with icon  
<i class="fa-solid fa-code"></i> Code icon  
<i class="fa-solid fa-chart-bar"></i> Chart icon
```

Icon Resources

- **Font Awesome:** fontawesome.com
- **Variety:** Thousands of free icons
- **Categories:** Technical, social, business, etc.

Usage Tips

- Use icons to enhance readability
- Maintain consistency in icon style
- Don't overuse - less is more



Step 13: Creating Interactive Modals

```
<!-- Trigger button -->  
<button type="button" class="btn btn-primary"  
      data-toggle="modal" data-target="#myModal">  
  Open Modal  
</button>
```

```
<!-- Modal structure -->  
<div class="modal fade" id="myModal" tabindex="-1">  
  <div class="modal-dialog modal-xl modal-dialog-centered">  
    <div class="modal-content">  
      <div class="modal-header">  
        <h5 class="modal-title">Modal Title</h5>  
        <button type="button" class="btn-close"  
          data-bs-dismiss="modal"></button>  
      </div>  
      <div class="modal-body">  
        Modal content goes here...  
      </div>  
    </div>  
  </div>  
</div>
```

Step 14: Adding Data Visualizations (1/3)

Chart Integration (Vega-Altair)

1. **Create Chart:** Generate JSON with Python/Altair
2. **Save File:** Place in `assets/charts/`
3. **Enable Vega:** Add `vega: true` to front matter
4. **Embed Chart:** Use shortcode in page

Step 14: Adding Data Visualizations (2/3)

Chart Implementation

```
# Python code to generate chart
import altair as alt

chart = alt.Chart(data).mark_bar().encode(
    x='column_a:N',
    y='column_b:Q'
).properties(width='container', height=300)

chart.save('assets/charts/my_chart.json')
```

Step 14: Adding Data Visualizations (3/3)

Embedding Chart in Jekyll

```
<!-- In your page -->  
<div style="height: 400px">  
  <vegachart schema-url="/repo-name/assets/charts/my_chart.json"  
    style="width: 100%; height: 100%"></vegachart>  
</div>
```

Step 15: Customizing Navigation

```
# In _config.yml
navbar:
  style: dual
  brand:
    title: "Your Project Title"
    img: "./assets/images/logo.png"
    url: /
  nav:
    - title: Guide
      url: /guide.html
    - title: Project
      url: /project.html
    ...
```

```
...
- title: Team
  url: /team.html
  - title: Resources
  child:
    - title: Documentation
      url: /docs.html
    - title: Downloads
      url: /downloads.html
```



Step 16: Theme Customization

Choosing a Theme

```
# In _config.yml
chulapa-skin:
  skin: "graymor" # or minty, lux, journal, etc.
```

Available Themes

- **graymor**: Professional gray theme
- **minty**: Fresh green theme
- **lux**: Elegant dark theme
- ...



Step 16: Theme Customization

Custom Colors

```
chulapa-skin:  
  vars:  
    primary-color: "#ff0000"  
    body-bg: "#fbf1ed"
```

Step 17: Adding Custom Fonts

Google Fonts Integration

```
# In _config.yml
googlefonts:
  - url: "https://fonts.googleapis.com/css2?family=Lekton:ital,wght@0,400;0,700;1,400&display=swap"

chulapa-skin:
  vars:
    headings-font-family: "Lekton"
```



Step 17: Adding Custom Fonts locally

Custom CSS Classes

```
// In assets/css/custom.scss
.custom-font {
  font-family: 'Lekton', monospace;
  font-weight: 400;
}

.custom-font-bold {
  font-family: 'Lekton', monospace;
  font-weight: 700;
}
```



Step 18: Advanced Layout Techniques

Full-Width Sections

```
<div class="full-width-wrapper">  
    
</div>
```

Step 19: Team Member Management

Team Configuration

```
# In _data/members.yml
- name: First Name Last Name
  github: https://kdd.isti.cnr.it/people/student-1
  role: student
```

Step 20: Final Checklist and Best Practices

Pre-Launch Checklist

-  **Repository Name:** Correct format (gX-2025-website)
-  **Baseurl:** Matches repository name
-  **Front Matter:** All pages have proper headers
-  **Navigation:** Updated and functional
-  **Team Info:** Complete member information
-  **Multimedia:** Images and videos properly placed
-  **GitHub Pages:** Deployment successful

Step 20: Final Checklist and Best Practices

Quality Assurance

- **Cross-browser Testing:** Chrome, Firefox, Safari
- **Mobile Responsive:** Test on different screen sizes
- **Load Speed:** Optimize images and assets
- **Accessibility:** Alt text for images, proper headings
- **SEO:** Meta descriptions, page titles

Thank You!

Questions?

- Review the guide documentation
- Check GitHub repository issues
- Consult with team members
- Refer to Jekyll community resources

Let's start building!